A Scalable Big Data Platform for Real-Time Urban Air Quality Tracking and Public Safety Alerting

Complete Project Report

23AID302 — Big Data Analytics

Hadoop · HDFS · Spark · YARN · Spark Structured Streaming · Hive · Scala · PySpark

154,583,979 rows · 33 GB · 241 cities · 4-node cluster

Every figure in this report is a measured value from a real, logged run.
The log backing each one is preserved in `docs/run_logs/`.

Report generated 26 September 2026

1. What the project does

Air pollution kills, and it kills on a timescale where an hour of warning matters. This project builds the full data platform behind a public early-warning service: it learns from thirteen years of Central Pollution Control Board station history, watches live atmospheric data for 230 Indian cities, forecasts each city's $PM_{2.5}$ one hour ahead, and issues a warning wherever the forecast National AQI crosses 150.

The deliverable is not a chart. It is a message:

> “Air quality in Jaisalmer is expected to reach AQI 500 (Severe) by 00:00 today, caused by coarse dust. Affects healthy people; seriously impacts those with existing disease. What to do: avoid all outdoor activity. Seek care if breathing is difficult.”

Everything else in this report — the 154-million-row lakehouse, the gradient-boosted model, the streaming engine, the Hive layer — exists to produce that sentence accurately and an hour early.

1.1 Starting point versus finishing point

	At the start	Now
Data	27.48 GB structured CSV only	33 GB: structured + two semi-structured layers, unified
Rows	146,276,379	154,583,979 in one queryable table
Model	None on disk; an sklearn reproduction quoted	Real Spark GBT, 300 trees, 241 cities, saved to HDFS
Streaming	Code written, never executed	Executed on the cluster; 1,130 live alerts fired
Serving	Nothing	Hive: 3 external tables, 3 views, 6 verified queries
Interface	Nothing	Public warning board, published and openable offline
Phases done	0–3 (partially)	0–10, all executed with preserved logs

The headline result

On a chronological hold-out scored on **2,074,569 real station-hours**, the forecaster reaches **R^2 0.8910 / RMSE 21.806 µg/m³** against a persistence baseline of **R^2 0.8587 / RMSE 24.831** — a **12.18% reduction in error**. The project's own quality gate passes by +0.0323 R^2.

2. Architecture: three data layers, three jobs

The three layers are not three copies of the same thing. Each exists because the others cannot do its job.

Layer	Size	Its one job	Why only it can
Structured CPCB 2010–2023	146,276,379 rows 27.48 GB	TRAIN the model	Only it has the label and thirteen years of seasonality
Semi-structured CAMS history 2022-08→	8,263,440 rows 458 MB	VALIDATE the structured data	Only independent source covering the same cities and hours
Semi-structured CAMS live	44,160 rows 6.7 MB	ALERT people now	Alerting is about now; history cannot warn anybody

2.1 How they join

Physically, in one Parquet table partitioned by `source_format`. Any row's origin is recoverable from `data_source`, so a query can use one layer, two, or all three.

source_format	data_source	rows	time range	cities
structured_csv	synthetic	130,023,448	2010-01 → 2023-03	241
structured_csv	real	16,252,931	2010-01 → 2023-03	241
semistructured_json	cams_history	8,263,440	2022-08 → 2026-09	230
semistructured_json	live_api	44,160	rolling 8 days	230
TOTAL		**154,583,979**		

This union is the project's Variety claim doing actual work: two genuinely different data models — flat fixed-schema CSV and nested self-describing JSON — read by their native Spark readers in one job and reconciled onto one schema, with neither reformatted at source.

2.2 The end-to-end flow

```
fetch CAMS + weather → LiveEnrichment (Spark on YARN) → 32-col CSV in HDFS
    → AirQualityStream (10 s micro-batches) → GBT forecast → CPCB AQI
    → alerts in HDFS → Hive serving layer → public warning board
```

3. Phase by phase: what was done and what came out

Phase 0 — Dataset construction and validation

Rebuilt from a 23.44 GB / 48.6M-row / 139-column CPCB source covering 453 real stations. All 16,252,931 real rows are kept verbatim; eight synthetic sibling stations per real station are generated with an AR(1) autocorrelated perturbation (per-station ϕ, $\sigma = 0.08$), copying each real station's missingness pattern rather than densifying it, and clamping every value to its physical range.

Output: 27.48 GB, 146,276,379 rows, 4,077 stations (453 real + 3,624 synthetic), 241 cities, 31 states/UTs, 2010–2023, 23 CSV parts. Five statistical fidelity tests pass.

Phase 1 — HDFS ingest

Output: raw CSVs, the clean layer, the feature layer, the unified table and the models, all on a 1 NameNode + 4 DataNode cluster. `hdfs fsck` reports HEALTHY on every path: 0 corrupt, 0 missing, 0 under-replicated blocks.

Four DataNodes rather than the minimum three is deliberate: with replication 3 on three nodes every node holds every block, so nothing actually distributes and node failure cannot be demonstrated.

Phase 2 — Spark and YARN activation

Output: Spark 3.5.6 running inside the cluster network, and YARN with a ResourceManager plus 4 NodeManagers (`yarn node -list` → Total Nodes: 4, all RUNNING). The live enrichment job was submitted to YARN as a real application, not local mode.

Phase 3 — Clean, range-guard, impute

Reads with an explicit string schema (never `inferSchema`), repairs the four measured source defects, range-guards the seven AQI pollutants plus solar radiation and pressure, then applies three-tier median imputation (station×month×hour → city×month → global). Critically, `pm25_measured` preserves the pre-imputation truth so the forecast label is never built from a filled value.

Output: 10.0 GB Snappy Parquet in HDFS, partitioned by city, 241 partitions, 100% of rows retained.

Phase 4 — Feature engineering at full scale

Ran for 73.5 minutes and reproduced the project's own RUNBOOK figures to the row:

Stage	Rows	Retained
Loaded from the clean layer	146,276,379	—

Stage	Rows	Retained
After dedupe on (station, hour)	128,660,967	87.96%
With a valid T+1 label	93,216,105	72.45%
Assemblable into a complete vector	93,216,105	100.00%

Output: 19.7 GB, 241 city partitions, **86-dimensional** feature vector (54 numeric + 32 one-hot). The T+1 label requires `next_ts == ts + 3600`; rows without a genuine successor are dropped rather than mislabelled.

Phase 5–6 — Model training and evaluation

GBT on the *delta* target (the change, not the level), chronological split, 300 trees, depth 6, step size 0.08. Ran 5 h 50 m and finished clean — the previous full-scale attempt had died at 3 h 50 m with a StackOverflow.

Metric — real rows only (n = 2,074,569)	R^2	RMSE	MAE
Persistence baseline (“next hour = this hour”)	0.8587	24.831	9.937
GBT forecaster	0.8910	21.806	8.926
Improvement	**+0.0323**	**-12.18%**	**-1.011**

All-rows figure (n = 18,671,121): R^2 0.8884 vs 0.8563, RMSE 22.429 vs 25.443, -11.85%. The script prints the persistence baseline on the same test rows every run and fails loudly if the model does not beat it.

Honest caveat: training used a seeded 10,000,552-row sample of the 74.5M-row training window because 74.5M × 86 dimensions will not fit in 15.7 GB of RAM. **Evaluation was not sampled** — all 18,671,121 test rows were scored, so the metric stands.

Phase 7–8 — Streaming speed layer and alert engine

Spark Structured Streaming, 10-second trigger, watching an HDFS directory. Each micro-batch rebuilds the Phase 4 features against a persisted 30-hour history table (so lag features survive job restarts), loads the GBT, predicts the $PM_{2.5}$ delta, converts to CPCB AQI, and writes an alert with severity and dominant pollutant wherever the forecast exceeds AQI 150.

Batch	rows in	predicted	alerts	latency
0 (replay)	120	15	10	10,750 ms
1 (replay)	120	15	10	7,817 ms
2 (replay)	120	14	10	7,019 ms
3 (replay)	120	12	7	6,894 ms
LIVE (after Phase 8b)	**16,560**	**16,560**	**1,130**	**23,452 ms**

Phase 8b — Live enrichment (the gap that had to be closed)

The first streaming run was fed rows already in the structured 32-column schema. That is a replay, not live consumption. Measured against the model's requirements, raw CAMS JSON supplies only 28 of the 54 numeric features:

Missing from raw CAMS JSON	n	Real source used
weather: temperature, humidity, wind speed and direction, rain, pressure, solar	8	Open-Meteo weather API (same provider, free, no key)
derived from weather: ventilation, scrub factor, stagnation, temp×humidity, is_raining	5	computed once weather exists

Missing from raw CAMS JSON	n	Real source used
traffic_index, is_rush_hour, urbanization_index	3	city_profiles.csv — the same profiles the generator used
*_was_imputed provenance flags	10	0 where the API returned a value, 1 where it returned null

`LiveEnrichment.scala` joins CAMS JSON, weather JSON and city profiles into the schema the stream already consumes, and was submitted to YARN as `application_1788670113534_0002`. Two unit conversions were measured, not assumed: CO from µg/m³ to mg/m³ (÷1000 — unconverted, structured CO averages 0.95 against CAMS 566), and pressure from hPa to mmHg (×0.750062 — the output 753.81 matches the structured median of 753). Wind is requested from the API in m/s directly.

Output: 16,560 live city-hours across 230 cities, zero nulls on model-critical columns, all 16,560 scored, **1,130 alerts**.

Phase 9 — Hive serving layer

Three EXTERNAL tables and three views over the Parquet already in HDFS — no data copied, and `DROP TABLE` removes only metadata, never 154 million rows. Exposed through `spark-sql` and, via `start-thriftserver.sh`, over the HiveServer2 Thrift protocol on port 10000, so beeline and JDBC clients connect unchanged.

All six demonstration queries return real results. The one worth showing is Delhi's seasonal cycle from measured values only:

Month	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
$PM_{2.5}$	180.5	127.9	88.6	86.8	79.5	66.3	44.0	38.8	47.1	120.1	213.6	199.4

A **5.5× swing** between the August monsoon low and the November stubble-burning peak. That is the physical reason thirteen years of history is required to train the model, and why a live feed — however fresh — cannot replace it.

Phase 10 — Public warning board

A single self-contained page. The architecture is the production shape, not a demo shortcut: an export job runs against the Hive layer and materialises one pre-aggregated JSON document, which is embedded into the page at build time. The page never queries 154 million rows, has no external dependency, and works opened from disk, hosted, or published.

It leads with the warning message each city's residents receive, carries CPCB's own health advisory wording for the band, and lets a reader search for their own city. Behind that sit the map of all 230 cities, warnings per hour, the cause breakdown, and the strongest warnings verbatim.

4. What the live system is producing

From the most recent cycle: **91 cities under warning**, 1,130 warnings issued across 230 monitored cities.

Severity	warnings
MODERATE	896
POOR	144
SEVERE	77
VERY POOR	13

Dominant cause	warnings	share
Ozone (O_3)	922	82%

Dominant cause	warnings	share
Coarse dust (PM_{10})	139	12%
Fine particles ($PM_{2.5}$)	67	6%
Sulphur dioxide	2	<1%

> **A finding the platform surfaced**
>
> **82% of current warnings are driven by ozone**, not particulates — a September photochemical-smog signal rather than a dust or combustion one. This is visible only because the alert engine records the dominant pollutant per alert, and it changes what a public warning should say: “avoid midday exertion” is the right advice for ozone, not “wear a mask”.

The worst forecast in the window is Jaisalmer at AQI 500 (Severe), driven by coarse dust — a desert city, which is consistent. The Punjab–Haryana–Delhi belt dominates the leaderboard.

5. Independent check: CPCB stations versus CAMS reanalysis

The eight-month overlap (Aug 2022 – Mar 2023) is the only window in which two independent sources describe the same cities over the same hours. Structured rows were averaged to a city-hour — CAMS gives one point per city, CPCB gives many stations — using measured values only, and joined on (city, timestamp): **1,152,707 matched city-hours**.

Pollutant	correlation	bias	RMSE	mean CPCB	mean CAMS
$PM_{2.5}$	0.565	+8.55	50.75	59.96	51.41
PM_{10}	0.558	+44.89	97.18	121.82	76.93
O_3	0.361	-48.54	67.17	28.54	77.07
NO_2	0.219	-2.07	35.24	24.87	26.94
SO_2	0.083	-10.05	43.32	14.60	24.65

This is a genuinely mixed result and is reported as one. Particulates track moderately, which is plausible for roadside sensors against a coarse model grid — a constant bias with decent correlation is the expected shape, not a failure. **SO_2 shows essentially no agreement**, and O_3 carries a large systematic offset. Per-city spread is wide: Kolkata 0.792 and Howrah 0.794 at the top, Pune -0.209 at the bottom.

Two methodological notes travel with those numbers:

- **CO was excluded** from the headline because the two sources report it in different units (mg/m³ versus µg/m³); comparing raw would manufacture a ~1000× disagreement.
- **A synthetic-versus-CAMS comparison was run and then discarded as invalid.** It returned figures numerically identical to real-versus-CAMS (0.564 against 0.565) because averaging to city-hour means averages away the AR(1) perturbation. The test cannot distinguish the two by construction, so it is not evidence either way; generator fidelity is tested at station level by the paired validator instead.

Timezone alignment was verified, not assumed. Correlation was measured at hour offsets -3…+3; it peaks at 0h/+1h (0.565 against 0.566 — within noise) and decays symmetrically either side. A genuine timezone error would shift the peak sharply.

6. Eleven defects found — and how each was caught

Four were in the source dataset. Seven were in pipeline code that had been written, and in several cases described as complete, but never executed. None surfaced until the code was run against real data on the cluster.

6.1 In the dataset

#	Defect	Scale	Fix
1	is_rush_hour / variation_id not binary	24,725 and 29,918 distinct values instead of 2 and 3	round, cast to int
2	T+1 label built with a positional lead()	8.69% of labels carried the wrong forecast horizon	require next_ts == ts + 3600 s
3	physically impossible values	humidity 104.999%, traffic_index 110.2 on a 0–100 scale	clamp to [0, 100]
4	sparse simultaneous AQI coverage	only 43.18% carry all 7 pollutants; 15.72% carry none	three-tier median imputation, every fill flagged

Defect 2 is the consequential one. It, not a modelling limit, was the root cause of an earlier run that scored $R^2 = 0.58$ — below the do-nothing persistence baseline of 0.81.

6.2 In the pipeline — found only by running it

#	Where	What happened	How it was caught
1	merge job	A substring file-name join double-counted Mumbai and Noida (navi_mumbai.json contains mumbai.json as a literal substring)	row count 44,544 failed to reconcile against the known fetch count of 44,160
2	streaming engine	Referenced ts_unix one line before it existed; every batch aborted at the history update	alerts printed BEFORE the failure, so it looked like a stall; only fires on a batch that has history
3	streaming engine	VectorAssembler handleInvalid="skip" silently dropped every live row, because CAMS returns ammonia as null for 100% of this region	16,560 rows in, 0 predicted, and no error at all
4	paths.py	HDFS join produced hdfs:////namenode:9000/… — four slashes; every HDFS-mode Python run would fail to find its own data	printed the resolved FEATURE_PATH inside the container
5	Phase 4 write	DataNodes went unresponsive under host memory pressure; with 4 nodes at replication 3 there was no spare node for the write pipeline	the first full-scale run computed everything correctly, then died writing 19.7 GB
6	Spark image	numpy absent, so PySpark's ML module aborts at import	training failed on first launch
7	Hive DDL	Declared a column named "timestamp" that does not exist; the real columns are observed_at / forecast_for / generated_at	counts and AQI looked correct while every time value came back NULL
8	live feeds	CAMS pulled in August joined against weather pulled in September shares zero hours	the join produced 0 rows from two healthy inputs

> **The pattern worth naming**
>
> Defects 3 and 7 share a shape: both produced **plausible-looking output with no error**. Sixteen thousand rows in and zero out, with the surrounding counts looking fine; a Hive table returning correct counts alongside NULL timestamps. A silent wrong answer is more dangerous than a crash, because nothing asks you to look. The habit that caught both — reconcile the output count against an independently known number — is the single most transferable practice in this project.

7. A request that was declined, and why

The project was asked for semi-structured coverage spanning 2010–2023, to match the structured layer. That was tested rather than assumed:

Date range requested	What the API actually returned
before 2013-01-01	rejected outright: “out of allowed range”
2013-01 → 2022-07	timestamps returned, but EVERY value null (0% fill)
2022-08	89.7% fill — real data begins here
2022-09 onward	~99.8% fill (ammonia 0%, a real regional gap)

Fulfilling the request literally would have produced roughly ten years of empty rows that *look* like coverage — the kind of thing an examiner finds immediately. Coverage therefore begins at the first month carrying real values, and the limitation is stated rather than hidden. Evidence: `_audit/cams_history_availability.py` and `cams_history_cutoff.py`.

Similarly, 230 of 241 cities resolved. Eleven (Bileipada, Byrnihat, Chikkaballapur, Davanagere, Eloor, Gummidipoondi, Kunjemura, Mandi Gobindgarh, Palkalaiperur, Sivasagar, Vatva) returned a genuine geocoder miss and were excluded rather than assigned invented coordinates.

8. The five V's

V	Evidence	State
Volume	154,583,979 rows, 33 GB, 1 NameNode + 4 DataNodes with real block distribution, fsck HEALTHY	demonstrated
Variety	flat fixed-schema CSV and nested self-describing JSON, read by native Spark readers into one 17-column schema; Hive queries span both	demonstrated
Veracity	11 defects found and fixed; independent cross-validation against CAMS reported honestly including where it is weak; every fill flagged	demonstrated
Velocity	Structured Streaming executed on the cluster: 10 s micro-batches, ~7 s latency, 1,130 alerts on genuinely live data	demonstrated
Value	R^2 0.8910 against persistence 0.8587 on 2,074,569 real test rows — an hour of warning before pollution arrives	demonstrated

9. Limitations, stated plainly

- **88.9% of structured rows are synthetic.** They are statistically validated against real data, but they are not physical measurements. Every model result is therefore reported on real rows separately, and that is the figure quoted.
- **Synthetic rows are easier to predict than real ones** (persistence R^2 0.856 against 0.733), so a blended headline would flatter the model.
- **Training used a 10M-row sample** of 74.5M available rows — a 15.7 GB RAM ceiling, not a pipeline limit. Evaluation was not sampled.
- **CAMS is a reanalysis model, not a ground sensor**, and ammonia is null across the whole region.
- **The semi-structured layers cover 230 of 241 cities** and begin in August 2022.
- **Cross-validation agreement is moderate for particulates and poor for SO_2 and O_3.**
- **Small-cluster fragility is real.** With 4 DataNodes at replication 3, a large derived write failed under memory pressure and had to be re-run at replication 2 with tuned timeouts. Replication 3 is kept for the raw layer, where fault tolerance is actually demonstrated.
- **The dashboard shows the most recent completed cycle**, not a continuously refreshing feed; the refresh is a scheduled job, not a push.

10. What exists on disk

10.1 Data

Path	Contents
airquality_expanded/part-*.csv	structured layer, 23 files, 27.48 GB, 146,276,379 rows
airquality_expanded/semistructured_history/	8,263,440 CAMS records, 230 city JSON files
airquality_expanded/live_cams/ + live_weather/	the live feeds, refreshed each cycle
airquality_expanded/unified_merged/	154,583,979 rows, Parquet, partitioned by source_format
HDFS /user/hadoop/	raw, clean_scala (10 GB), features (19.7 GB), unified, models, stream output

10.2 Code

File	What it does
scripts/00c_expand_dataset.py	builds the structured dataset; repairs the four source defects
scripts/00d_validate_expansion.py	five paired statistical fidelity tests
scripts/00e_fetch_semistructured_json.py	live CAMS pull for every city in city_profiles.csv
scripts/00f_fetch_semistructured_history.py	historical CAMS pull; resumable, rate-limit aware
scripts/00g_fetch_live_weather.py	live weather pull (8 fields, m/s requested explicitly)
scripts/01_clean_select.py	Phase 3: range-guard, three-tier imputation, pm25_measured
scripts/01c_merge_structured_semistructured.py	unions structured CSV and nested JSON onto one schema
scripts/02_feature_extraction.py	Phase 4: T+1 label, CPCB AQI, lags, rolling, interactions
scripts/03_train_gbt.py	Phase 5–6: chronological split, persistence gate, GBT on delta
scripts/04_cross_validate_overlap.py	CPCB vs CAMS over the overlap window
scripts/run_daily_live_cycle.py	the operational job: fetch → upload → enrich → hand off → dashboard
scala/…/CleanSelect.scala	Scala port of Phase 3 for on-cluster execution
scala/…/FeatureExtraction.scala	Scala port of Phase 4
scala/…/AirQualityStream.scala	Phase 7–8: streaming speed layer and alert engine
scala/…/LiveEnrichment.scala	Phase 8b: CAMS + weather + profiles → model schema, on YARN
hive/create_tables.sql, demo_queries.sql	Phase 9: external tables, views, six verified queries
dashboard/	Phase 10: export from Hive, build, and the warning board itself
scripts/_audit/ (15 scripts)	every quoted number is reproducible from these

The Scala ports exist for a concrete reason: Spark cannot spawn Python worker processes on the development machine, so anything that must run on the cluster is written for the JVM.

10.3 Documents and evidence

File	Contents
docs/PROJECT_REPORT.pdf	this document
docs/DATASET_REPORT.docx / .pdf	the dataset and platform reference, 13 sections

File	Contents
docs/ARCHITECTURE.md	how the three layers combine; the live-alerting design
docs/WORK_LOG.md	every claim tied to the script and log that produces it
docs/HADOOP_SPARK_CHEATSHEET.pdf	one-page start/stop card for the cluster
docs/run_logs/ (10 logs)	raw stdout from every real run — the primary evidence

11. How to run it

Start the cluster, then one command does the daily cycle:

```
docker compose -f docker-compose.yml -f docker-compose.yarn.yml -f docker-compose.spark.yml up -d
python scripts/run_daily_live_cycle.py --all
```

That fetches both live feeds in the same window (fetching them separately once produced zero overlapping hours), uploads to HDFS, runs LiveEnrichment on YARN, hands the file to the streaming job's input directory, waits for it to be scored, and rebuilds the warning board. The streaming engine is a separate always-on service; dropping the file into its directory is the whole handoff.

12. Conclusion

The platform does what its title says. Thirteen years of real CPCB history train a forecaster that beats the honest baseline by 12.18% on two million real test hours. Live atmospheric data flows through a Spark-on-YARN enrichment job into a Structured Streaming engine that issues an hour's warning wherever the forecast AQI crosses 150. Everything the platform produces is queryable through Hive and visible on a public warning board.

What the project demonstrates beyond the architecture is a working method: measure rather than assume, reconcile every output count against an independently known number, and report the weak results as plainly as the strong ones. Eleven defects were found that way — several of which produced confident, plausible, wrong answers and would have survived any amount of code review.

All figures in this report are measured. Logs in `docs/run_logs/`; reproduction scripts in `scripts/_audit/`.